

RESEARCH & ENGINEERING INTERNSHIP REPORT

Design and Implementation of a Low-Cost Modular Telemetry System

An ESP32-Based Edge Platform with a Custom Event Processing Backend

INTERN:	—
INSTITUTE:	Tata Institute of Fundamental Research (TIFR)
SUPERVISOR:	—
DATE OF SUBMISSION:	July 24, 2026

ABSTRACT

This report presents the design and implementation of **EZTNOW Thermal**, a low-cost modular environmental telemetry system developed using ESP32-based edge sensor nodes, Sensirion SHT40 temperature and humidity sensors, and a custom event-processing backend.

The system enables distributed environmental monitoring by collecting sensor measurements at edge devices and transmitting telemetry data over a local wireless network to a centralized processing system. A major component developed as part of this project is **MAWMAW**, a modular telemetry and event-processing engine designed to decouple data ingestion, processing, and application-specific outputs through an extensible architecture.

The complete platform integrates embedded firmware, wireless communication, backend event processing, Prometheus-based metrics collection, and Grafana visualization into a unified monitoring pipeline. The project explores practical challenges in embedded systems development, sensor interfacing, network deployment, and scalable telemetry architecture.

The resulting system demonstrates a flexible foundation for low-cost environmental monitoring applications, while providing an extensible architecture for future additions such as additional sensors, processing pipelines, and automated analysis capabilities.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my internship supervisor, —, for providing me with the opportunity to undertake this project at the Tata Institute of Fundamental Research (TIFR). His guidance, technical discussions, and constructive feedback throughout the internship were invaluable in shaping both the direction and execution of this work.

Contents

1	Introduction	1
1.1	Motivation & Background	1
1.2	Project Objectives	1
1.3	Scope of the Project	2
1.4	Overview of EZTNOW Thermal	2
1.5	Report Organization	3
2	System Architecture	4
2.1	Design Requirements	4
2.2	Overall System Architecture	5
2.3	Data Flow Pipeline	5
2.4	Deployment Architecture	5
3	Hardware Design and Sensor Integration	7
3.1	ESP32 Edge Node	7
3.2	SHT40 Temperature and Humidity Sensor	8
3.3	Hardware Assembly and Prototyping	9
4	Embedded Firmware Development	11
4.1	Development Environment	11
4.2	Firmware Architecture	11
4.3	Telemetry Data Format	11
4.4	Deployment Considerations	12
5	MAWMAW Telemetry Engine	13
5.1	Motivation and Design Goals	13
5.2	Architecture	13
5.3	Performance Characteristics	15
6	Monitoring Infrastructure	16
6.1	Prometheus Integration	16
6.2	Grafana Dashboard	17
6.3	Python Metrics Exporter	17
7	Future Work	18
7.1	Authentication	18
7.2	ESP-MESH	18
7.3	InfluxDB and Integration with Existing Pipelines	18
7.4	Hardware Expansion	19
7.5	Predictive Analytics	19
7.6	Alerting and Notifications	19
8	Conclusion	20

1 INTRODUCTION

1.1 MOTIVATION & BACKGROUND

Environmental parameters such as temperature and humidity are critical indicators of system health across a wide range of applications, including electronic infrastructure, manufacturing environments, storage facilities, and distributed monitoring systems. Continuous telemetry of these parameters enables the early detection of abnormal conditions and provides valuable information on the behavior of the system over time.

At TIFR, several High Performance Computing (HPC) clusters are deployed across multiple server racks. These systems support the computational requirements of different research groups, including large-scale data analysis and scientific computing. Since the racks are located within the same facility, continuous environmental monitoring is important for ensuring reliable operation of the computing infrastructure.

Although individual racks provide internal monitoring, ambient temperature and humidity measurements surrounding the racks are not integrated into the existing monitoring infrastructure. Independent environmental measurements provide an additional layer of fault tolerance by allowing ambient conditions to be cross-referenced with the internal sensor readings. Such information can also be incorporated into alerting systems and assist in routine monitoring and maintenance.

Furthermore, these rack systems are supplied by external vendors and are typically accompanied by proprietary monitoring and visualization platforms. These platforms are often expensive, closed, and difficult to integrate with existing software tools. As a result, the collected data cannot be readily incorporated into custom analysis pipelines, research workflows, or machine learning models. Although the vendor-provided monitoring systems are designed to be reliable, their measurements cannot be independently verified or correlated with external environmental sensors.

Conventional monitoring solutions therefore often rely on specialized hardware platforms and tightly integrated ecosystems. Although these systems provide reliable functionality, their cost and limited flexibility can become significant constraints when deploying large numbers of monitoring nodes.

This motivates the development of a low-cost modular telemetry architecture in which individual sensing nodes can be deployed at scale while maintaining centralized processing, visualization, and extensibility. Such a system requires both efficient embedded hardware and a software architecture capable of handling heterogeneous data sources and future expansion.

1.2 PROJECT OBJECTIVES

The primary objectives of this project were:

- To design a **low-cost environmental telemetry system** capable of collecting temperature

and humidity data from distributed edge devices.

- To develop an **embedded sensing platform** using inexpensive hardware and digital environmental sensors for reliable data acquisition and wireless transmission.
- To design and implement a **modular telemetry processing engine** capable of receiving, processing, and distributing sensor events through extensible interfaces.
- To integrate **real-time monitoring** capabilities through metrics collection and visualization tools.
- To create an architecture that supports **future expansion** through additional sensors, processing modules, and edge devices.
- To implement a **locally operated** telemetry infrastructure that does not depend on external cloud services for data collection and visualization.

1.3 SCOPE OF THE PROJECT

The major components within the scope of this work include:

- **Embedded Sensing:** Development of inexpensive edge nodes capable of acquiring temperature and humidity measurements using digital sensors.
- **Telemetry Communication:** Implementation of wireless data transmission between edge devices and a central processing system over a local network.
- **Backend Processing:** Design and development of a modular telemetry engine for event ingestion, processing, and integration with external monitoring systems.
- **Monitoring Infrastructure:** Integration of metrics collection and visualization components for real-time observation of environmental data.
- **System Integration:** Evaluation of the complete pipeline from sensor acquisition to dashboard visualization.

The current implementation focuses on local WiFi-based telemetry between edge nodes and a centralized processing system. Features such as long-range mesh communication, advanced analytics, and production-scale deployment infrastructure are outside the scope of this project. However, the modular architecture developed provides a foundation for future extensions, including additional communication protocols, larger sensor networks, and intelligent data processing capabilities.

1.4 OVERVIEW OF EZTNOW THERMAL

EZTNOW Thermal is an end-to-end environmental telemetry system designed to collect, process, and visualize environmental measurements from distributed sensing nodes. The system consists of three primary layers: an **embedded sensing layer** responsible for data acquisition, a **telemetry processing layer** responsible for event handling and data transformation, and a **monitoring layer** responsible for metrics collection and visualization.

The embedded layer collects environmental measurements from ESP32 sensor nodes and transmits telemetry data over a local wireless network. The processing layer is implemented using **MAWMAW**, a modular event-processing engine developed as part of this project to provide

a flexible interface between incoming telemetry data and external applications. The monitoring layer integrates metrics collection and visualization tools such as Grafana and Prometheus to provide real-time observation of system conditions.

The complete architecture was designed with emphasis on modularity, low deployment cost, and extensibility, allowing additional sensors, communication methods, and processing capabilities to be incorporated in future iterations.

1.5 REPORT ORGANIZATION

The remainder of this report is organized as follows. Chapter 2 presents the overall system architecture and design requirements. Chapters 3 and 4 describe the hardware platform and embedded firmware, respectively. Chapter 5 discusses the design and implementation of the MAWMAW telemetry engine, while Chapter 6 covers the monitoring infrastructure and system integration and Chapter 7 outlines potential future work.

2 SYSTEM ARCHITECTURE

2.1 DESIGN REQUIREMENTS

Based on the objectives established in Section 1.2, a set of functional and non-functional requirements was identified to guide the system design. These requirements influenced the selection of hardware components, communication mechanisms, and software architecture.

Requirement	Description
Low Cost	The sensing node should be constructed using inexpensive, readily available hardware components.
Modularity	Individual system components should be independently replaceable or extendable without major architectural changes.
Local Operation	The complete telemetry pipeline should operate without requiring cloud-based services or Internet connectivity.
Real-Time Telemetry	Sensor measurements should be transmitted and processed with sufficiently low latency for continuous monitoring.
Scalability	The architecture should support the addition of multiple sensing nodes without requiring redesign of the processing pipeline.
Extensibility	Support for additional sensors, communication protocols, and processing modules should be straightforward.
Maintainability	The software architecture should separate hardware interaction, telemetry processing, and visualization to simplify future development.

Table 1: Primary engineering design requirements.

2.2 OVERALL SYSTEM ARCHITECTURE

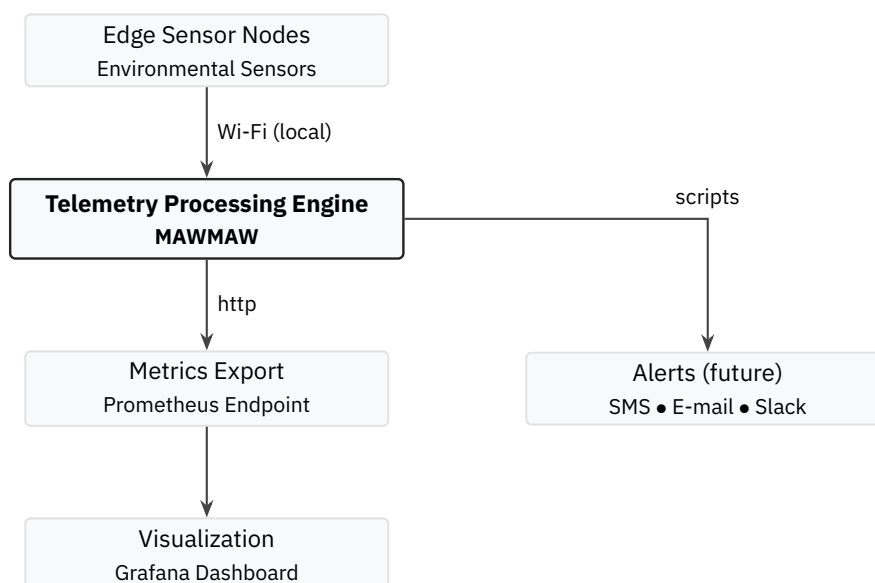


Figure 1: Overall architecture of the EZTNOW Thermal telemetry system.

2.3 DATA FLOW PIPELINE

Temperature and humidity measurements are periodically (every second for this prototype) acquired by the edge sensing node. After local preprocessing, the measurements are transmitted over Wi-Fi to the central telemetry processing engine. Upon reception, the data are converted into internal events and processed by MAWMAW's event pipeline, using runtime loadable python scripts. These events are subsequently exposed through a Prometheus-compatible metrics endpoint, allowing Grafana to retrieve and visualize the measurements in real time. The modular event-processing architecture also enables additional scripts to consume the same event stream for custom processing, automation, or integration with external systems.

2.4 DEPLOYMENT ARCHITECTURE

The prototype was designed as a locally deployed telemetry system requiring only a **single host computer and one or more wireless sensing nodes**. The host computer performs multiple roles simultaneously, acting as the wireless access point, telemetry server, metrics collector, and visualization platform.

Each edge sensing node periodically acquires environmental measurements and transmits them over the local wireless network to the central host. The telemetry processing engine executes on the host computer, where incoming measurements are converted into events, processed, and exposed as Prometheus metrics. Grafana, running on the same system, retrieves these metrics for real-time visualization through configurable dashboards.

By consolidating the server components onto a single machine, the prototype eliminates the need for external cloud infrastructure or dedicated networking equipment. Additional sensing

nodes can be incorporated by connecting them to the same wireless network without requiring modifications to the backend architecture.

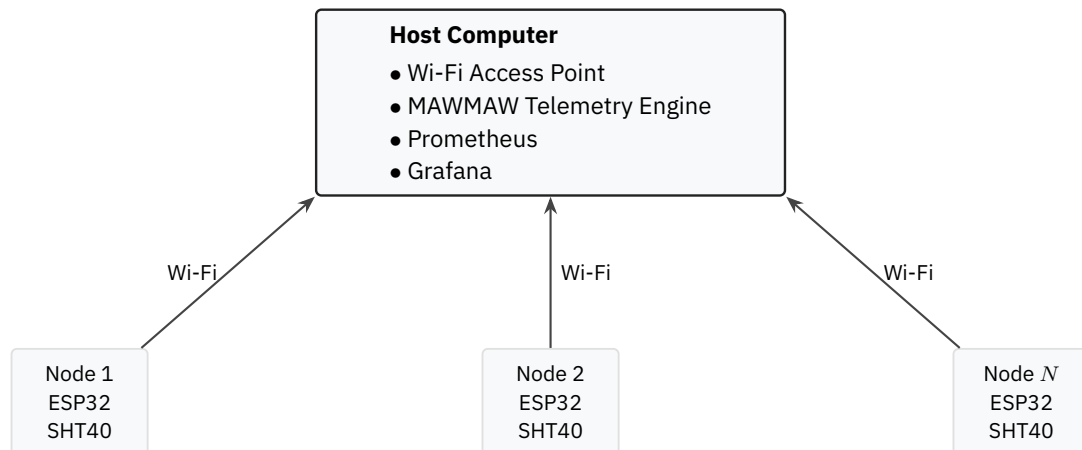


Figure 2: Deployment architecture of the EZTNOW Thermal prototype.

3 HARDWARE DESIGN AND SENSOR INTEGRATION

3.1 ESP32 EDGE NODE

The edge sensing node forms the interface between the physical environment and the telemetry infrastructure. Its primary responsibilities are to periodically acquire measurements from the environmental sensor, perform basic preprocessing, and transmit telemetry to the central processing system over a wireless network. As a result, the selected embedded platform was required to provide an appropriate balance between computational capability, wireless connectivity, power consumption, development flexibility, and cost.

Several embedded platforms were investigated during the hardware selection process.

The Arduino Uno is widely adopted for rapid prototyping and educational applications due to its simplicity and extensive software ecosystem. However, the platform does not provide integrated wireless networking and therefore requires additional communication modules for remote telemetry. This increases both hardware complexity and deployment cost while offering limited processing resources for future expansion.

The Raspberry Pi Pico W provides a modern ARM-based microcontroller with integrated wireless networking and substantially greater computational capability than traditional Arduino boards. Although suitable for embedded applications, its ecosystem is primarily centered around general microcontroller development. In comparison, the ESP32 offers a more mature software ecosystem for wireless IoT systems together with broader support for networking libraries and development frameworks.

The ESP8266 was also considered because of its low cost and integrated Wi-Fi capability. While well suited for simple Internet of Things applications, its comparatively limited memory, single-core processor, and reduced peripheral support restrict future extensibility. Since this prototype was intended to serve as a flexible platform for continued development, these limitations made it less suitable as the primary edge device.

The ESP32 provides integrated Wi-Fi and Bluetooth connectivity, a dual-core processor, multiple communication interfaces, low-power operating modes, and a mature software ecosystem while remaining inexpensive and widely available. Furthermore, support for both the Arduino framework and the ESP-IDF development framework enables rapid prototyping during initial development while allowing migration to lower-level software when greater control over the hardware is required.

Based on these considerations, the ESP32 was selected as the edge computing platform for the prototype. Its combination of integrated wireless connectivity, computational capability, low cost, and software support best satisfied the engineering requirements established in Chapter 2.

However it is worth noting that the ESP8266 fairs similarly if development ease is not taken into account.

Feature	Arduino Uno	Pico W	ESP8266	ESP32
Integrated Wi-Fi	External Module	Built-in	Built-in	Built-in
Bluetooth	Not Available	Built-in	Not Available	Built-in
Processor	8-bit AVR	Dual-Core ARM Cortex-M0+	Single-Core Xtensa LX106	Dual-Core Xtensa LX6
RAM	2 KB	264 KB	160 KB	520 KB
Flash Memory	32 KB	2 MB	4 MB	4 MB
Power Consumption	20–45 mA	1.1–100+ mA	0.02–380 mA	0.005–260 mA
Development Ecosystem	Mature	Mature	Mature	Mature
ESP-IDF Support	Not Supported	Not Supported	Not Supported	Supported
Typical Cost (INR)	Rs. 500–800	Rs. 450–700	Rs. 250–350	Rs. 300–400
Overall Suitability	Fair	Good	Very Good	Excellent

Table 2: Comparison of embedded platforms considered for the edge sensing node.

3.2 SHT40 TEMPERATURE AND HUMIDITY SENSOR

Environmental measurements were acquired using the **Sensirion SHT40 digital temperature and humidity sensor**. The device integrates both sensing elements within a single package and communicates using the Inter-Integrated Circuit (I²C) protocol, allowing both measurements to be obtained through a common digital interface.

Unlike analog temperature sensors, the SHT40 performs **signal conditioning and analog-to-digital conversion internally**. As a result, the microcontroller receives calibrated digital measurements directly, reducing susceptibility to electrical noise and eliminating the need for external calibration circuitry.

The sensor measures both ambient temperature and relative humidity, enabling a single sensing module to monitor multiple environmental parameters simultaneously. In addition, the device incorporates an internal heater that can be activated to reduce the effects of condensation or persistent high-humidity conditions. Although this functionality was not required for the present prototype, it provides flexibility for future deployments in more demanding environments.

Several low-cost digital environmental sensors were considered during the design process. The selection criteria included measurement accuracy, communication interface, integration complexity, and overall cost. Since the system was intended for continuous environmental monitoring, accurate and reliable measurements were prioritized while maintaining a low hardware cost.

Among the commonly available alternatives, the **DHT22** is widely used due to its simplicity and affordability. However, its **lower measurement accuracy and slower response characteristics** make it less suitable for applications requiring more reliable environmental telemetry. The SHT40 offers substantially improved measurement accuracy while remaining within a similar price range, making it a practical choice for the present system.

Feature	DHT22	SHT40
Temperature Accuracy	$\pm 0.5^{\circ}\text{C}$	$\pm 0.2^{\circ}\text{C}$
Humidity Accuracy	$\pm 2-5\% \text{ RH}$	$\pm 1.8\% \text{ RH}$
Interface	Single-Wire	I ² C
Temperature + Humidity	Yes	Yes
Integrated Heater	No	Yes
Typical Cost (INR)	Rs. 130-200	Rs. 150-200
Overall Suitability	Good	Excellent

Table 3: Comparison of environmental sensors considered for the prototype.

3.3 HARDWARE ASSEMBLY AND PROTOTYPING

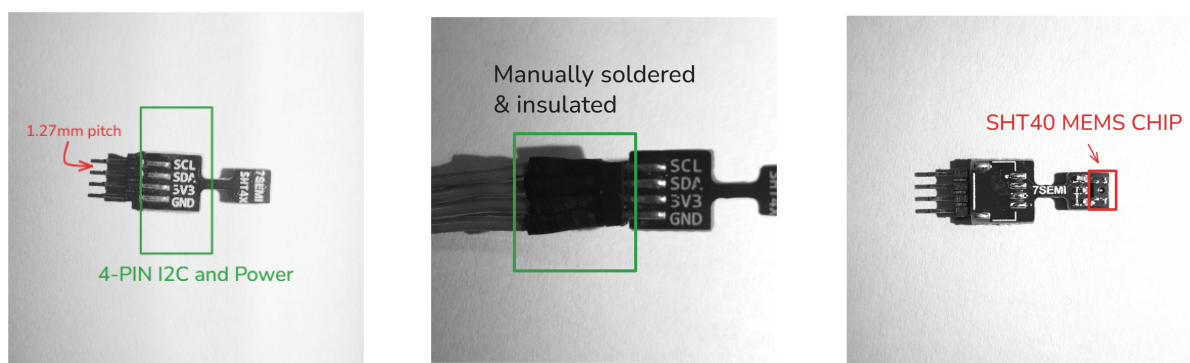
The wiring configuration used throughout the project is summarized in Table 4. Once assembled, the ESP32 periodically acquired temperature and humidity measurements from the sensor before transmitting the resulting telemetry to the central processing system over the local wireless network.

SHT40 Pin	ESP32 Pin
VCC	3.3 V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Table 4: Electrical connections between the SHT40 sensor and the ESP32 development board.

For hardware prototyping, the sensor was initially obtained as a compact breakout board exposing a **1.27 mm pin pitch**. Although electrically functional, this connector spacing was **incompatible with standard 2.54 mm Dupont jumper** wires commonly used during rapid prototyping, requiring manual soldering to establish reliable electrical connections.

To simplify future development, the corresponding Nano Breakout version with a 2.54mm Pitch Connector is recommended, as it exposes the same sensor through standard 2.54 mm headers while requiring no changes to the electrical interface or firmware.



(a) Rear view of the original breakout board showing the 1.27 mm connector footprint.

(b) Manual soldering required to interface the board with standard prototype wiring.

(c) Front view of the sensor module.

Figure 3: Hardware prototyping of the SHT40 breakout board. The original breakout exposes a 1.27 mm connector pitch, requiring manual soldering before integration with the ESP32 development board.

4 EMBEDDED FIRMWARE DEVELOPMENT

4.1 DEVELOPMENT ENVIRONMENT

Firmware development was carried out using **Visual Studio Code** together with the **PlatformIO** extension. PlatformIO simplifies embedded development by automating toolchain installation, dependency management, project configuration, firmware compilation, flashing, and serial monitoring.

The firmware was developed using the **Espressif Internet of Things Development Framework (ESP-IDF)**, the official software development framework for ESP32 devices. Although PlatformIO also supports the Arduino framework, ESP-IDF was selected because it provides lower-level access to the Wi-Fi stack, FreeRTOS scheduler, networking APIs, and peripheral drivers, enabling greater control over the system behaviour.

The `esp-idf-lib SHT4x` component was used to interface with the environmental sensor, providing a well-maintained driver while allowing development effort to focus on the telemetry application rather than low-level device communication.

4.2 FIRMWARE ARCHITECTURE

The firmware was designed as a modular application responsible for hardware initialization, environmental sensing, wireless communication, and telemetry transmission. To improve responsiveness and maintainability, these functions were separated into independent FreeRTOS tasks rather than being implemented within a single execution loop.

System execution begins in the `app_main()` entry point, where the non-volatile storage, sensor interface, and Wi-Fi subsystem are initialized. Once initialization is complete, the firmware creates dedicated tasks for network management and telemetry transmission.

The network management task maintains connectivity with the host access point, while the telemetry task periodically acquires measurements from the SHT40 sensor and transmits them to the central server as JSON-formatted messages over a TCP connection.

The complete firmware source code is available in the accompanying project repository [1]. Only the high-level architecture is presented in this report.

4.3 TELEMETRY DATA FORMAT

Environmental measurements acquired by the ESP32 are transmitted to the central processing system over a TCP connection. Each measurement is encoded as a lightweight JSON object containing the device identifier, temperature, relative humidity, and a Unix timestamp. The `device_id` is automatically generated from the ESP32's unique Wi-Fi MAC address and formatted as `ESP32_<MAC-ADDRESS>`, allowing individual sensing nodes to be uniquely identified without requiring manual configuration. JSON was selected because of its readability, widespread software support, and ease of parsing within the telemetry backend.

A typical telemetry message is shown below.

```
{  
  "device_id": "ESP32_84F3EB12AB34",  
  "temperature": 27.63,  
  "humidity": 58.14,  
  "timestamp": 1753024812  
}
```

Each telemetry message represents a single sensor measurement. Upon reception, the MAW-MAW telemetry engine parses the JSON payload, converts the measurement into an internal event, and forwards it through the event processing pipeline for metrics generation and further processing.

The message format was intentionally kept simple to simplify debugging during development while remaining sufficiently extensible to support additional sensor measurements or metadata in future revisions.

4.4 DEPLOYMENT CONSIDERATIONS

The prototype was developed for use within a trusted local network to simplify development and testing. As a result, the firmware currently uses a hard-coded Wi-Fi network identifier (SSID) (MAWMAW) and password (helloWorld) to establish connectivity with the host system.

In addition, telemetry messages are transmitted over a plain TCP connection without application-layer authentication or encryption. While this design simplifies deployment and debugging, it is intended solely for prototype development and should not be considered suitable for production environments.

A production deployment would require secure credential management, mutual device authentication, encrypted communication (for example, TLS), and appropriate access control mechanisms to protect both the sensing nodes and the telemetry infrastructure.

5 MAWMAW TELEMETRY ENGINE

5.1 MOTIVATION AND DESIGN GOALS

The telemetry requirements of EZTNOW Thermal required a backend capable of receiving measurements from multiple sensing nodes, processing incoming data with minimal latency, and exposing the resulting information to monitoring applications. Rather than implementing a purpose-built backend, the project integrates **MAWMAW**, a high-performance event processing engine developed prior to this project.

MAWMAW was designed around two primary objectives: **performance** and **modularity**. The engine employs an event-driven architecture with minimal processing overhead, enabling large numbers of events to be handled concurrently while maintaining **sub-millisecond processing latency** under typical workloads. This makes the engine suitable for real-time telemetry and other latency-sensitive applications.

The second major design objective is modularity. Instead of embedding application-specific functionality into the core engine, MAWMAW exposes a plugin interface through which protocol ingestors, data exporters, scripting engines, and application-specific processing modules can be integrated without modifying the core engine. As a result, new functionality can be added by developing independent plugins rather than altering the underlying event-processing infrastructure.

Within EZTNOW Thermal, MAWMAW serves as the central event processing engine. Incoming telemetry from the ESP32 sensing nodes is converted into internal events and subsequently processed by dynamically loadable plugins responsible for metrics generation, visualization, and other application-specific functionality. The internal architecture of the engine is described in the following section.

Further implementation details, including the complete source code and documentation of the MAWMAW engine, are available in the accompanying project repository [2].

5.2 ARCHITECTURE

MAWMAW follows a modular event-driven architecture in which protocol-specific functionality is separated from the core event processing engine. Incoming telemetry is received through dynamically loadable ingestor plugins, translated into a common internal event representation, and dispatched by the core engine to independent processing modules.

This design decouples network communication from application logic, allowing the same event stream to be consumed simultaneously by multiple plugins. Within EZTNOW Thermal, an access-point ingestor receives telemetry transmitted by the ESP32 sensing nodes and forwards each measurement to the MAWMAW core. From there, events are processed by scripts responsible for bounds checking, cleaning, and export. Finally the data is exported to Prometheus via a `http endpoint`

Concurrency is achieved through independent execution contexts for event ingestion and script

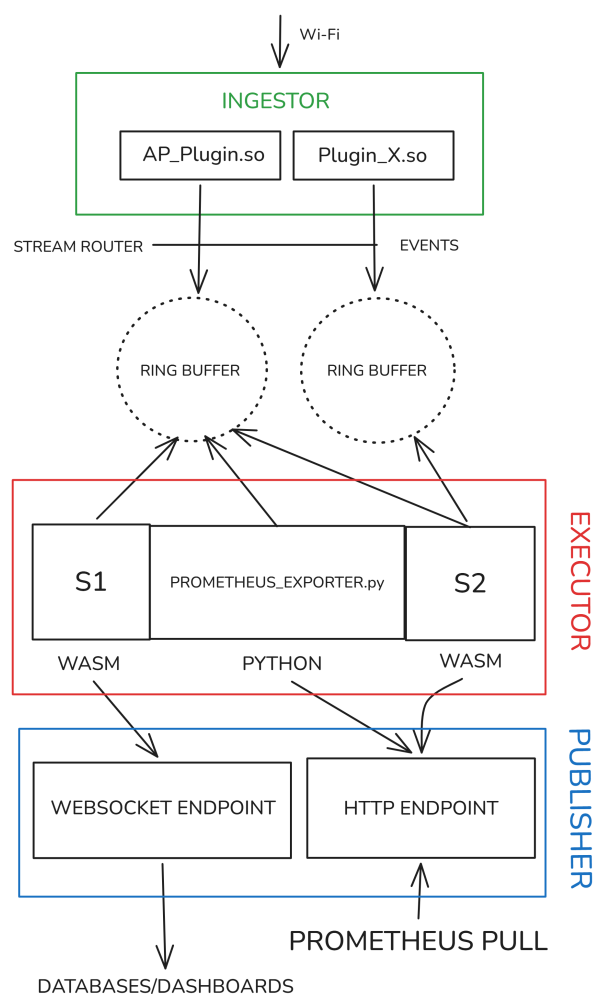


Figure 4: Internal architecture of the MAWMAW event processing engine. Incoming telemetry is received through protocol-specific ingestor plugins, converted into internal events, processed by runtime-loadable scripts, and exported to monitoring systems through independent plugins.

execution, allowing multiple telemetry sources and processing modules to operate simultaneously. Runtime-loadable plugins further simplify maintenance by enabling new functionality to be added without recompiling the engine itself.

The engine is intentionally protocol-independent. Rather than embedding support for specific communication standards within the core, protocol handlers are implemented as plugins. This approach enables the same event processing infrastructure to support multiple telemetry sources while maintaining a compact and lightweight core.

The overall organization of the engine is illustrated in Figure 4. The modular design separates protocol handling, event processing, and data export into independent components, allowing new functionality to be added through plugins without modifying the core engine.

5.3 PERFORMANCE CHARACTERISTICS

The primary objective of this project was to develop and demonstrate a functional telemetry architecture rather than perform a comprehensive performance evaluation. As a result, detailed benchmarking of the MAWMAW engine was outside the scope of the present work.

During development and testing, the complete telemetry pipeline operated reliably on modest hardware, including an older Intel Pentium Gold laptop with 4 GB of RAM running Rocky Linux. Under these conditions, end-to-end telemetry latency was typically observed to be on the order of sub-milliseconds for individual events while maintaining a low memory footprint.

Although these observations indicate that the architecture is suitable for real-time environmental monitoring, a formal performance analysis involving throughput, latency distributions, CPU utilization, memory consumption, and scalability under increasing numbers of sensing nodes remains future work.

6 MONITORING INFRASTRUCTURE

6.1 PROMETHEUS INTEGRATION

MAWMAW exposes telemetry through an HTTP endpoint compatible with the Prometheus text exposition format. Rather than communicating directly with the ESP32 nodes, Prometheus periodically scrapes this endpoint at a configurable interval and stores the exported metrics within its time-series database.

The telemetry engine therefore acts as a bridge between the event-driven backend and the monitoring infrastructure. Environmental measurements received from the ESP32 nodes are first processed internally before being converted into Prometheus metrics that can be queried and visualized using standard monitoring tools.

A typical request to the metrics endpoint is shown below.

```
$ curl http://localhost:9090/metrics

# HELP esp32_temperature_celsius Temperature reading
# TYPE esp32_temperature_celsius gauge
esp32_temperature_celsius{device_id="ESP32_AABBCC"} 28.81

# HELP esp32_humidity_percent Relative humidity
# TYPE esp32_humidity_percent gauge
esp32_humidity_percent{device_id="ESP32_AABBCC"} 73.12
```

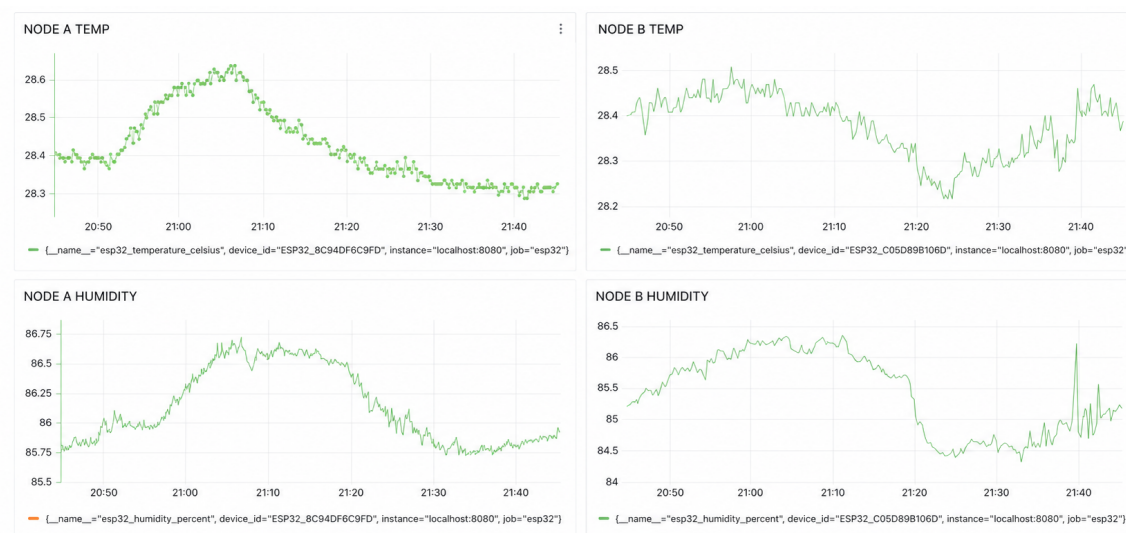


Figure 5: Grafana dashboard displaying live environmental telemetry.

6.2 GRAFANA DASHBOARD

Grafana was used to visualize the metrics exported by MAWMAW through Prometheus. After configuring Prometheus as the data source, dashboards were created to display temperature and humidity measurements in real time for each sensing node.

The dashboard allows multiple ESP32 devices to be monitored simultaneously while retaining historical measurements for later inspection. Since the telemetry is exposed using the standard Prometheus interface, additional dashboards and alerting rules can be created without modifying the telemetry engine itself.

6.3 PYTHON METRICS EXPORTER

A Python processing script was developed for MAWMAW to convert incoming telemetry events into Prometheus-compatible metrics.

Each ESP32 periodically transmits an independent JSON packet containing its current measurements. Since individual devices operate asynchronously, packets may arrive in any order and at slightly different times. As a result, simply forwarding each event directly to Prometheus would result in incomplete metric snapshots.

To address this, the exporter maintains an **in-memory registry** indexed by the unique device identifier. Whenever a new telemetry event is received, the corresponding device entry is updated with the latest temperature, humidity, timestamp, and packet count. The exporter then regenerates the complete Prometheus text exposition from the current registry state before serving it through the HTTP endpoint.

Maintaining the latest state for every device ensures that the exported metrics always represent a consistent snapshot of the monitored system, independent of the order or timing in which telemetry packets arrive. The exporter also removes stale devices after a configurable timeout and records diagnostic metrics such as packet counts and JSON parsing errors.

Adding support for alerts from here would be trivial, however it is best left to another script, as each script spawns its own thread.

7 FUTURE WORK

7.1 AUTHENTICATION

The prototype was developed for operation within a trusted local network and therefore employs a simple pre-shared Wi-Fi network configuration using a fixed SSID and password. Telemetry is transmitted over a plain TCP connection without additional device authentication or payload encryption in order to simplify development and integration.

For production deployments, stronger security mechanisms such as mutual device authentication, encrypted communication (e.g., TLS), and secure credential management should be incorporated to prevent unauthorized access and protect telemetry data.

7.2 ESP-MESH

The current prototype assumes that all ESP32 sensing nodes are located within the wireless coverage of a single access point, which is typically sufficient for deployments spanning approximately 30 m under normal indoor conditions. This was considered adequate for the intended deployment within the HPC facility at TIFR, where all monitored racks are located within a single room.

For larger deployments, the ESP32 platform also supports **ESP-MESH**, in which intermediate nodes can forward telemetry on behalf of more distant devices. Such a multi-hop topology can significantly extend network coverage without requiring additional wireless infrastructure. Since the anticipated deployment area did not require coverage beyond a single access point, ESP-MESH functionality was not incorporated into the present prototype.

7.3 INFLUXDB AND INTEGRATION WITH EXISTING PIPELINES

Although the prototype exports telemetry using the Prometheus exposition format, the modular architecture is not restricted to a single monitoring backend. Additional exporter plugins can be developed to forward the same event stream to alternative time-series databases such as **InfluxDB**, enabling long-term storage, historical analysis, and integration with existing monitoring infrastructures.

This flexibility is particularly valuable in research environments where multiple data processing pipelines may already be in use. Since MAWMW separates event processing from data export, the same telemetry can be simultaneously delivered to several independent consumers without modifying the embedded firmware. Consequently, environmental measurements can be integrated with existing dashboards, archival systems, or data analytics workflows while preserving a common telemetry source.

7.4 HARDWARE EXPANSION

Although the ESP32 was selected for the present prototype, the lower-cost **ESP8266** remains a practical alternative for future deployments. For applications involving only periodic sensor acquisition and telemetry transmission, its computational capabilities are generally sufficient while offering a lower hardware cost. However, the ESP32 was preferred during development because its larger memory, additional peripherals, dual-core architecture, and more mature software ecosystem simplified implementation and provided greater flexibility for future expansion.

The current prototype monitors only temperature and relative humidity, as these parameters are the primary environmental indicators for data center operation. Nevertheless, future work may investigate the inclusion of additional sensors, such as atmospheric pressure, air quality, or volatile organic compound (VOC) sensors, to determine whether correlations exist that could improve anomaly detection or provide earlier indications of equipment faults.

A battery-powered version of the sensing node may also be considered for deployments where mains power is unavailable. However, within a data center environment, power is readily accessible and continuous operation is expected. Consequently, the additional cost and maintenance associated with battery-powered operation were not considered justified for the present application.

7.5 PREDICTIVE ANALYTICS

The telemetry architecture developed in this project provides a foundation for future machine learning workflows aimed at predictive maintenance and automated fault detection. In addition to ambient temperature and humidity measurements, future deployments may integrate internal telemetry from server hardware, including processor temperatures, coolant temperatures, fan speeds, power consumption, and other diagnostic information. Since MAWMAW is capable of aggregating heterogeneous event streams, these measurements can be synchronized within a common telemetry pipeline for subsequent analysis.

Such datasets would enable the development of anomaly detection and predictive maintenance models capable of identifying abnormal operating conditions before conventional threshold-based alarms are triggered. Depending on the quantity and quality of the collected data, techniques ranging from statistical models and tree-based methods to recurrent neural networks or transformer-based time-series models could be investigated. The modular architecture presented in this work allows these inference pipelines to be incorporated with minimal changes to the underlying telemetry infrastructure.

7.6 ALERTING AND NOTIFICATIONS

The telemetry pipeline developed in this project can be readily extended to support automated notifications through services such as e-mail, SMS, Slack, or other messaging platforms. Since MAWMAW processes telemetry as an event stream, plugins or scripts can evaluate incoming measurements against predefined conditions and generate alerts whenever abnormal operating

conditions are detected.

Although this functionality was considered during the design phase, it was not implemented within the scope of the present work due to time constraints. However, The modular architecture allows such notification mechanisms to be incorporated with minimal changes to the existing telemetry pipeline.

8 CONCLUSION

This project implemented an inexpensive modular telemetry system to monitor ambient environmental conditions around the HPC racks at TIFR. By combining ESP32 microcontrollers and Sensirion SHT40 sensors with the MAWMAW event processing engine, the system delivers precise temperature and humidity metrics without depending on proprietary vendor software or cloud services.

The prototype validates the complete data path from hardware acquisition and local network transport to stateful metric aggregation and Grafana visualization. Overall, the system achieves all primary design goals, providing a reliable local monitoring baseline that can easily adapt to future additions like mesh networking, automated alerts, and analytics.

REFERENCES

- [1] kyuQee, “Eztnow thermal: Data center thermal monitor using mawmaw esp32+sht40,” <https://github.com/kyuQee/EZTNOW-thermal>, 2026, gitHub repository.
- [2] —, “Mawmaw: High-performance telemetry and event processing engine,” <https://github.com/kyuQee/mawmaw>, 2026, gitHub repository.